

Streamlit 프론트엔드와 시스템 아키텍처

개발자 전용이던 API에 '얼굴'을 달아준다 — 드래그 한 번이면 비개발자도 쓰는 화면으로

오늘의 한 문장 — Streamlit은 파이썬 스크립트가 곧 웹앱입니다. 단, 클릭 한 번마다 스크립트가 통째로 다시 실행되므로 비싼 객체는 캐시로, 결과는 기억(session_state)으로 지켜야 하죠. 모델은 이미 완성한 FastAPI가 들고 있습니다 — 화면은 그 API를 부르는 클라이언트일 뿐입니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

화면을 만든다

Streamlit으로 `.py` 스크립트 하나를 웹 UI로 띄우고, **클릭마다 전체가 재실행되는** 실행 모델을 설명할 수 있다.

02

캐시와 기억을 쓴다

두 캐시(`cache_data` / `cache_resource`)를 구분해 재실행 비용을 끊고, `session_state`로 재실행에도 결과를 지킨다.

03

구조를 나눈다

프론트(8501)와 백엔드(8000)를 **왜** 분리하는지 — 독립 배포·다양한 클라이언트 — 이유를 댈 수 있다.

04

돌을 잇는다

업로드한 이미지를 **Base64 + POST**로 FastAPI에 보내고, 실패까지 친절히 안내하는 대시보드를 완성한다.

API는 완성됐다 — 그런데 누가 쓰지?

Day 1~3으로 추론 API는 끝났습니다. 그런데 이 서비스를 **비개발자**에게 건네면 어떻게 될까요?

Day 1 — 모델을 파일로 굳혔습니다. 학습한 모델을 저장하고, 추론 로직을

`predict()` 함수로 분리해 뒀죠. 모델은 이제 노트북 밖으로 나갈 준비가 되었습니다.

Day 2 — 그 함수를 API로 감쌌습니다. FastAPI에 Pydantic 검증까지 붙여, 브라우저 문서 화면에서 호출과 테스트가 됩니다. 기계라면 누구든 부를 수 있는 입구가 생긴 거예요.

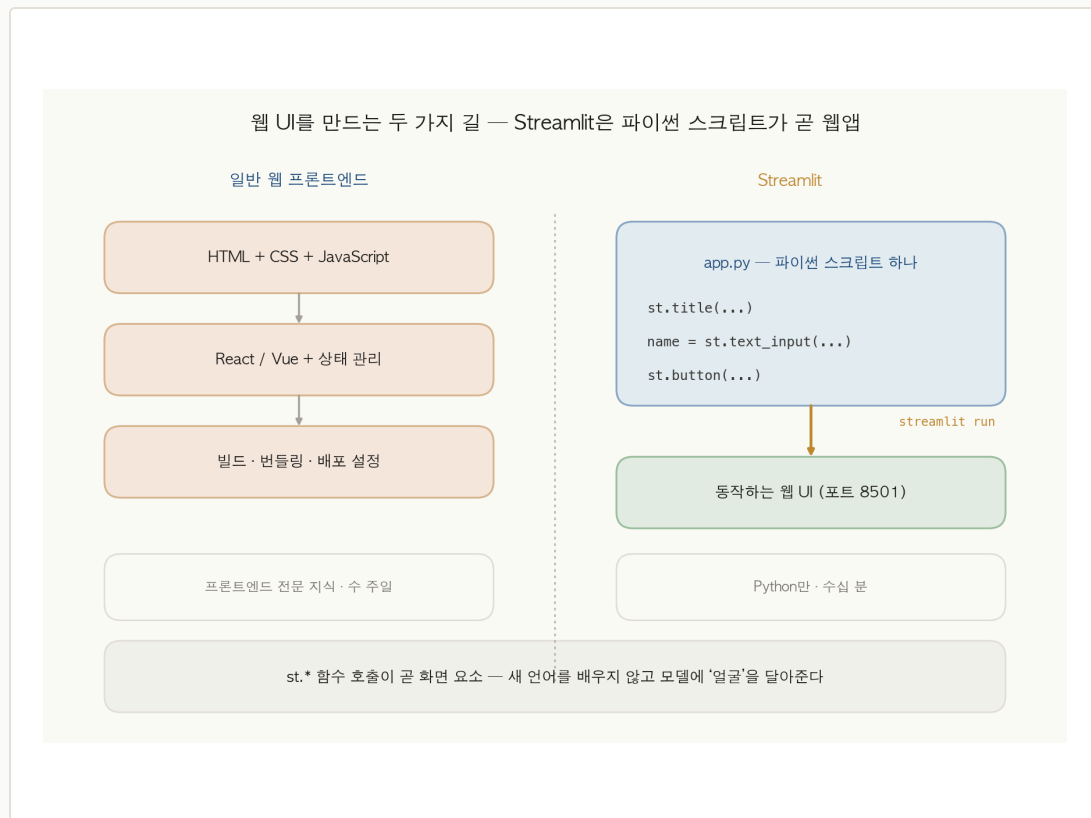
Day 3 — 그 API를 튼튼하게 만들었습니다. 무거운 추론 분리·전역 에러 처리·기록(logging)으로, 동시 요청과 예외에도 죽지 않는 서버가 되었습니다. 백엔드는 이걸로 완성입니다.

그런데 입구가 개발자 전용입니다. "문서 화면 열고 JSON을 직접 적으세요"라는 안내는 비개발자에게엔 무리예요. 사용자가 원하는 건 **이미지를 드래그하면 답이 나오는 화면**입니다.

오늘은 그 화면을 만듭니다. 프론트엔드를 처음 다루지만 새 언어는 안 배웁니다 — Python만으로 끝나요. 그리고 백엔드는 **한 줄도 안 고칩니다.** 이미 완성된 API를 '부르기만' 할 겁니다.

Streamlit — 파이썬 스크립트가 곧 웹앱이다

Streamlit은 **프론트엔드 지식 없이** Python 스크립트 하나로 인터랙티브한 웹 화면을 만드는 도구입니다. HTML·CSS·JavaScript가 전혀 필요 없어요 — `st.button()` 을 부르면 버튼이, `st.image()` 를 부르면 이미지가 **그냥 그려집니다**.



01 일반 웹 UI는 비싸다

화면 구조(HTML)·꾸밈(CSS)·동작(JS)에 전용 프레임워크까지 — 전문 지식과 수 주일이 듭니다. 모델 데모 하나 보여주자고 치르기엔 큰 값이죠.

02 Streamlit은 호출이 곧 화면

`st.*` 함수를 부르는 게 곧 화면 요소를 그리는 일입니다. 스크립트 하나면 수십 분 안에 동작하는 UI가 나와요.

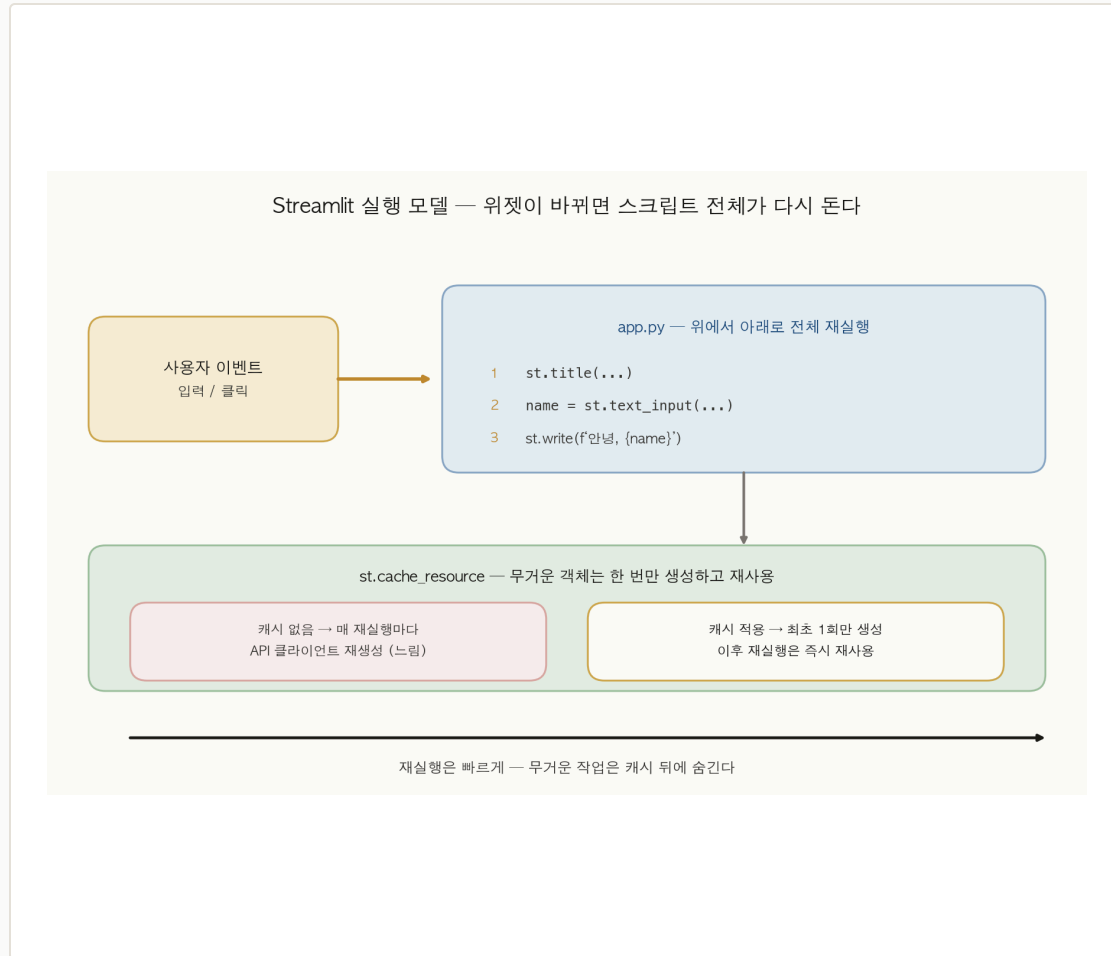
03 ML 엔지니어에게 딱 맞다

모델 데모를 빠르게 만들어 팀·고객에게 보여주고, 내부 대시보드도 직접 만듭니다. 프론트 개발자를 기다릴 필요가 없죠.

04 이 과정의 프론트엔드

이후 프로젝트(Day 5·7)의 화면도 Streamlit입니다. 오늘 익히는 위젯·캐시·API 호출 패턴이 그대로 재사용돼요.

클릭 한 번에 스크립트 전체가 다시 돈다



위젯 값이 바뀌면 `app.py`가 처음부터 끝까지 재실행된다 — 오늘 내내 이 한 문장이 따라다닌다.

01 일반 웹과 다르다

보통의 웹은 클릭에 해당하는 **그 부분만** 실행합니다. Streamlit은 클릭·입력 한 번에 **스크립트 전체를 위에서 아래로** 다시 실행해요. 화면 전체를 매번 새로 그리는 셈이죠.

02 위젯 값은 반환값으로

재실행되며 `st.text_input()` 이 **지금 입력된 값**을 돌려주고, 아래 코드가 그 값으로 다시 그려집니다. 복잡한 이벤트 연결 없이 평범한 스크립트가 인터랙션이 되는 비결입니다.

03 버튼은 '순간'이다

`st.button()` 은 **눌린 그 재실행에서만** True고, 다음 재실행에선 도로 False입니다. 그래서 추론 실행 같은 1회성 방어쇠에 쓰죠 — 결과를 남기려면 별도 저장이 필요해집니다(§5).

04 그래서 두 가지가 필요하다

비싼 객체를 그냥 두면 **매 재실행마다 다시 만들고**(→ §2 캐시), 변수에 담은 결과는 **재실행마다 사라집니다**(→ §5 `session_state`). 오늘 배우는 도구 둘 다 이 실행 모델의 보완책이에요.

17줄이면 첫 화면이 뜬다

직접 띄워 보죠. Streamlit 앱은 `.py` 파일로 저장해 터미널에서 `streamlit run` 으로 실행합니다 — 브라우저가 열리며 `localhost:8501` 에 화면이 뜹니다.

```
import streamlit as st

# 반드시 '첫 번째' Streamlit 호출이어야 한다
st.set_page_config(page_title="첫 앱", layout="centered")

st.title("나의 첫 Streamlit 앱")           # 큰 제목 (H1)
st.write("Python 코드만으로 만든 웹 화면") # 범용 출력
st.divider()                             # 수평 구분선

name = st.text_input("이름을 입력하세요:", placeholder="홍길동")
if name:                                  # 입력 → 전체 재실행 → 새 값
    st.success(f"안녕하세요, {name}님!")   # 초록 메시지
else:
    st.info("위에 이름을 입력해 보세요.")  # 파랑 메시지

# 실행: 터미널에서
# streamlit run frontend/app_hello.py --server.port 8501
```

01 set_page_config

탭 제목·아이콘·레이아웃을 정합니다. **반드시 스크립트의 첫 Streamlit 호출**이어야 해요 — 재실행 모델 특성상 페이지 설정은 맨 위에서 한 번만 정해져야 하기 때문입니다.

02 출력 함수 셋

`st.title` (큰 제목)·`st.write` (범용)·`st.divider` (구분선). 특히 `write` 는 문자열·dict·표 등 **타입을 보고 알아서** 그려 줍니다.

03 text_input = 재실행의 입구

이름을 입력하는 순간 §1.3의 재실행이 일어나고, `name` 에 새 값이 담겨 아래 `if` 가 다시 평가됩니다. "**스크립트가 다시 돌며** 변수가 새로 채워진다"는 감각을 여기서 익혀 두세요.

04 상태 메시지 4색

`success` (초록)·`info` (파랑)·`warning` (노랑)·`error` (빨강). §4에서 API 호출의 성공·실패를 사용자에게 색으로 알릴 때 그대로 씁니다.

React로도 되는데, 왜 Streamlit?

범용 도구와 특화 도구의 차이입니다 — 데이터·ML 화면이라는 일에는 맞는 쪽이 따로 있어요.

범용 웹

React · Vue

화면을 직접 짠다

구조·스타일·동작을 전부 손으로 작성합니다. 상태 관리와 화면 전환 설계도 개발자 몫이죠.

자유롭지만 비싸다

어떤 화면이든 픽셀 단위로 만들 수 있지만, 간단한 데모 하나에도 전문 지식과 시간이 듭니다.
"이미지 올리면 숫자 보여주기"에 빌드 설정부터 시작하는 셈이죠.

어울리는 곳

고객에게 내놓는 **제품 화면** — 세밀한 UX와 브랜드 디자인이 필요한 서비스에 강합니다.

데이터·ML 특화

Streamlit

위젯을 호출만 한다

`st.file_uploader()` 한 줄이면 드래그 앤 드롭 업로더가 그려집니다. HTML은 한 줄도 안 씁니다.

제약이 곧 속도

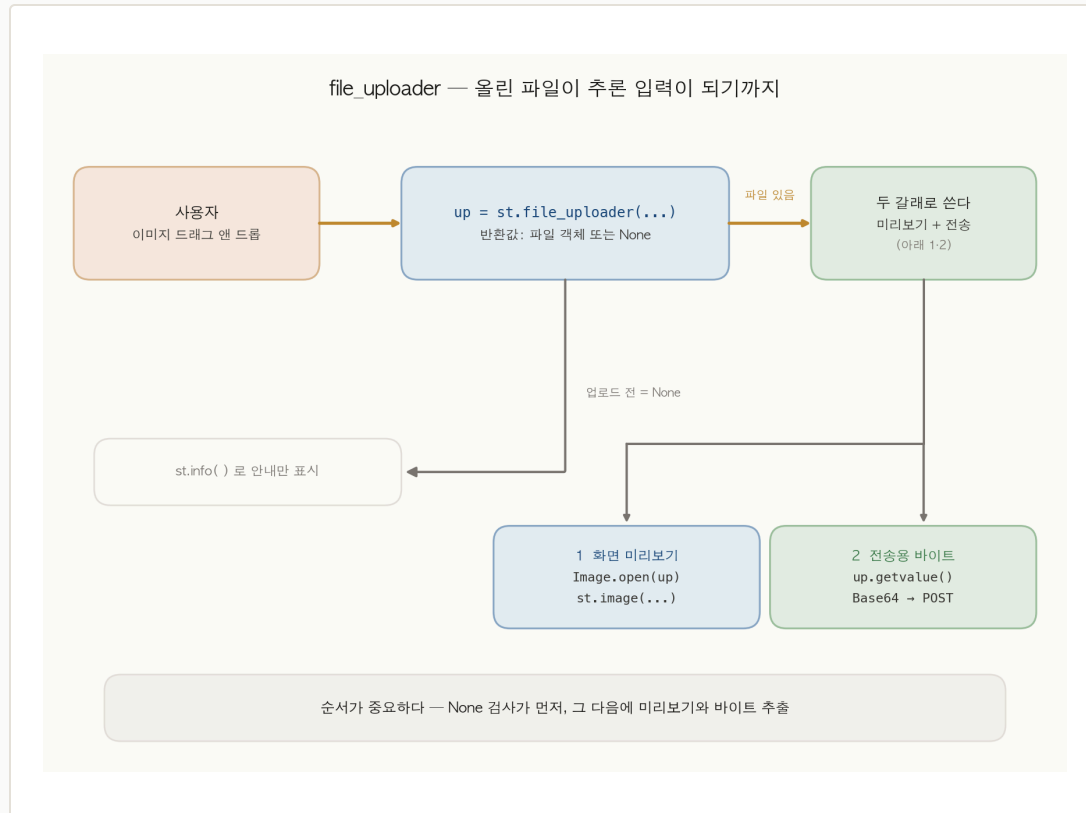
레이아웃 자유도는 낮지만, 데이터 앱·대시보드라면 혼자서 수십 분 만에 완성됩니다. 정해진 틀이 있어서 디자인 고민 자체가 사라지는 거예요.

어울리는 곳

모델 데모·내부 대시보드·프로토타입 — **Python 사용자가 혼자 빠르게** 만드는 화면에 최적입니다. 오늘의 MNIST 대시보드가 정확히 이 경우죠.

file_uploader가 주는 건 파일이 아니라 '객체 또는 None'

이미지 추론의 **입구**가 되는 위젯입니다. `st.file_uploader()` 는 업로드 전엔 `None` 을, 업로드 후엔 **파일 객체**를 돌려줘요. 이 객체에서 미리보기용 이미지와 전송용 바이트, **두 갈래**를 뽑아 씁니다.



01 None 검사가 먼저

앱이 처음 뜨면 아직 아무 파일도 없으니 반환값은 `None` 입니다.

`if up is not None:` 으로 감싸지 않으면 첫 화면부터 에러로 깨져요.

02 갈래 1 — 미리보기

`Image.open(up)` 으로 열어 `st.image()` 로 작게 띄웁니다. 사용자가 "내가 올린 게 맞네"를 눈으로 확인하게 해주는 단계죠.

03 갈래 2 — 전송용 바이트

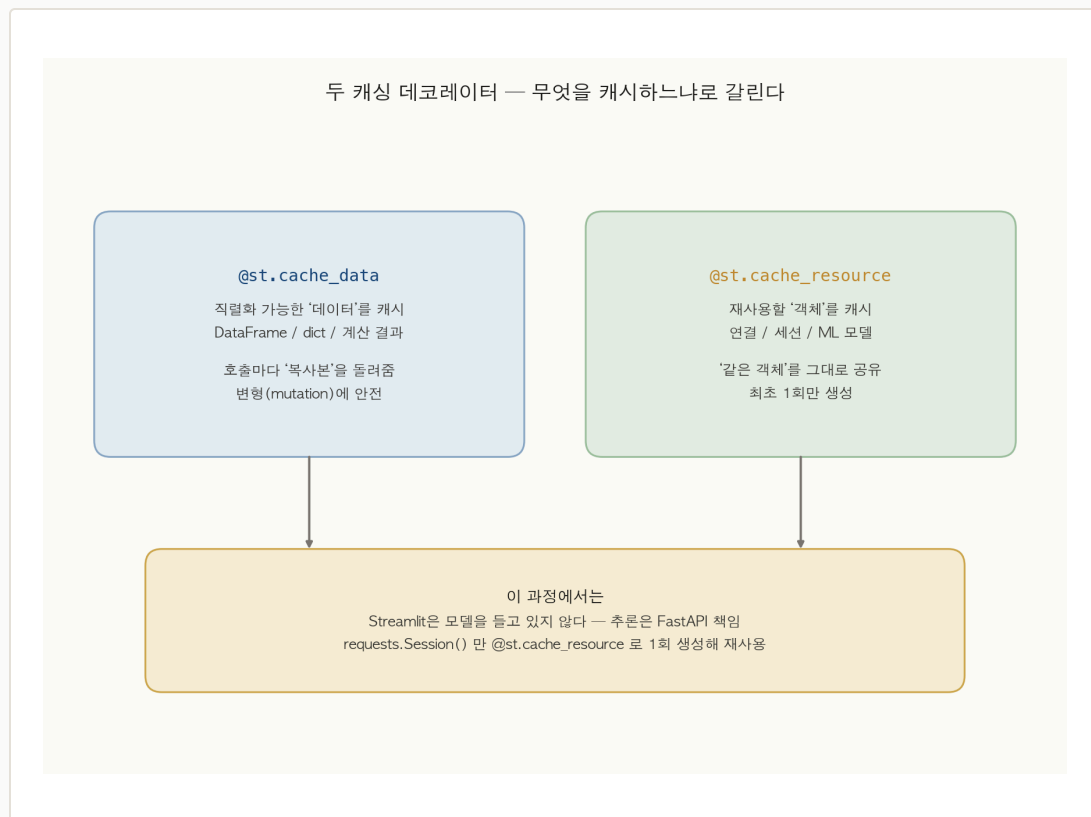
`up.getvalue()` 가 파일의 **원본 바이트**를 돌려줍니다. §4에서 이걸 Base64로 바꿔 FastAPI로 보내요 — 전처리엔 백엔드 몫, 프론트는 받은 그대로 전달만 합니다.

04 type=으로 1차 방어

`type=["png", "jpg"]` 로 확장자를 화면에서 먼저 거릅니다. 물론 **진짜 검증**은 Day 2에서 배운 대로 백엔드 Pydantic의 몫이에요.

cache_data나 cache_resource나 — 무엇을 캐시하느냐로 갈린다

재실행 모델의 첫 번째 청구서 — 비싼 것을 **매 재실행마다 다시 만들면** 앱이 느려져요. 그래서 캐시가 돌입합니다. `st.cache_data` 는 값으로 굳힐 수 있는 데이터에, `st.cache_resource` 는 살아 있는 객체에 — 함수 위 데코레이터 한 줄이면 됩니다.



01 cache_data — 데이터를 복사해 준다

표·dict·계산 결과처럼 **파일로 저장할 수 있는** 것에 씁니다. 호출마다 **복사본**을 돌려줘서, 받은 쪽이 값을 고쳐도 캐시 원본은 안전해요.

02 cache_resource — 같은 객체를 공유한다

DB 연결·HTTP 세션·ML 모델처럼 **복사하면 안 되는** 것에 씁니다. 최초 1회만 만들고, 이후엔 모두가 **동일한 객체**를 함께 써요.

03 헛갈리면 이렇게 묻자

"이걸 JSON 파일로 저장할 수 있나?" — 저장되면 데이터(cache_data), 안 되면 자원(cache_resource)입니다. 연결과 모델은 파일로 못 굳히죠.

04 인자가 캐시 열쇠

같은 인자로 부르면 캐시를 쓰고, 다른 인자면 새로 계산합니다. 참고로 이 과정에선 모델을 FastAPI가 들고 있어, 프론트가 캐시할 건 가벼운 HTTP 세션 정도예요.

추론은 어디서 돌릴까 — 화면 안? 화면 밖?

Streamlit으로 모델 추론을 보여주는 길은 두 가지입니다. 그리고 이 과정은 둘 중 하나를 분명하게 고릅니다.

방법 A — 한 덩어리(모놀리식). Streamlit 앱 안에서 직접 모델을 로드하고 추론합니다. 서버 하나, 파일 하나 — UI와 모델이 **한 프로세스**에 같이 사는 구조예요.

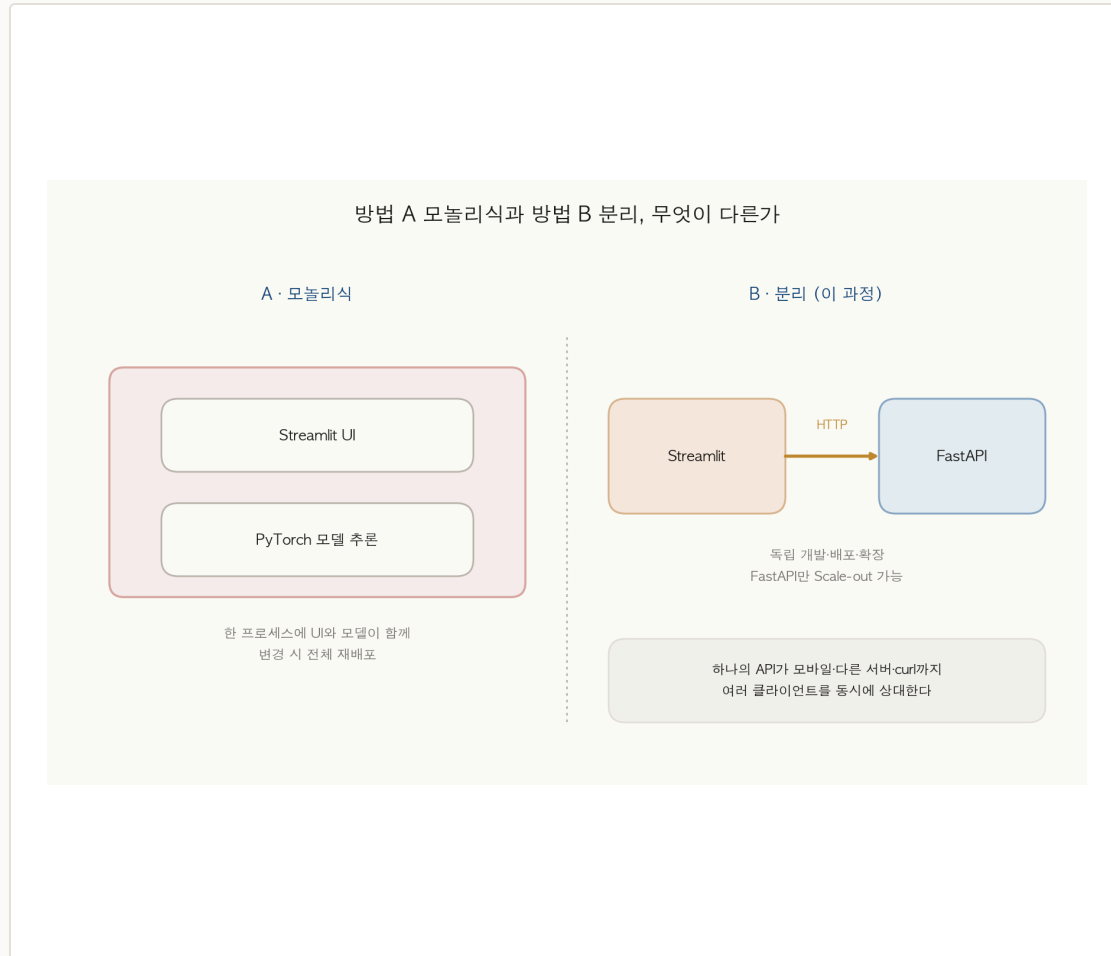
방법 B — 분리. Streamlit은 화면만 맡고, 추론은 **별도의 FastAPI 서버**에 HTTP로 요청합니다. 그 서버는 새로 만들 필요도 없어요 — Day 1~3에서 이미 완성한 그것입니다.

A가 더 간단해 보입니다. 띄울 서버도 하나, HTTP 통신도 없으니까요. 하루짜리 데모라면 실제로 A도 나쁘지 않습니다. 그런데 이 과정은 **B**를 택합니다.

A의 청구서는 금방 옵니다. 화면 문구 한 줄을 고쳐도 무거운 모델을 든 앱 **전체**를 다시 띄워야 하고, 추론 부하가 늘면 UI까지 통째로 복제해야 합니다. UI와 모델의 운명이 묶여버린 대가예요.

B는 이미 가진 자산을 재사용합니다. Day 3까지 만든 튼튼한 API가 그대로 백엔드가 되고, 오늘 새로 쓰는 건 **그 앞에 붙는 화면 한 겹**뿐입니다. 분리가 오히려 일을 줄여주는 거죠 — 왜 그런지 다음 장에서 구조로 봅니다.

한 덩어리로 둘까, HTTP로 나눌까



A는 UI와 모델이 한 프로세스, B는 HTTP로만 연결 — 이 과정은 B를 쓴다.

01 A — 한 덩어리

Streamlit 안에서 모델을 직접 로드·추론합니다. 시작은 단순하지만 UI와 모델이 한 몸이라, **작은 변경에도 전체 재배포**가 따라와요.

02 B — 분리

Streamlit은 UI, FastAPI는 추론 — 둘은 HTTP로만 대화합니다. 각자 따로 개발하고, 따로 배포하고, 따로 확장할 수 있어요.

03 부하가 말해주는 차이

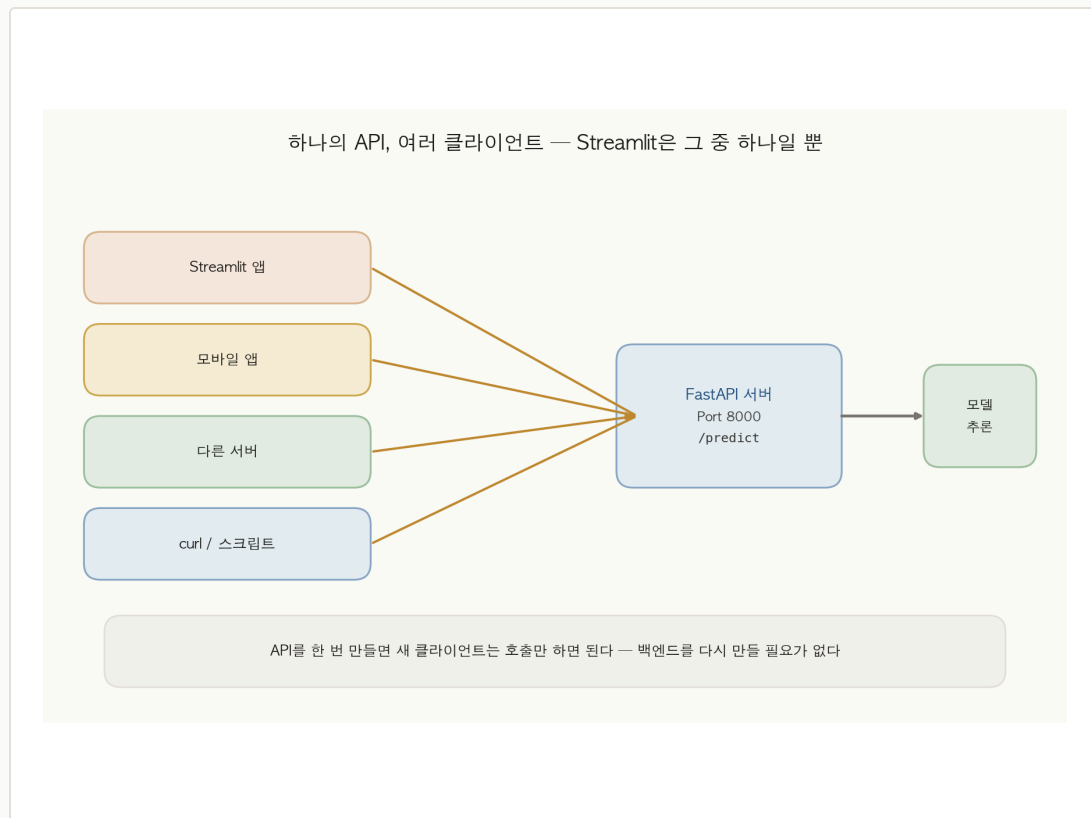
추론 요청이 몰리면 A는 UI까지 통째로 복제해야 하지만, B는 **FastAPI만** 여러 대로 늘리면 됩니다. 비싼 GPU 서버를 화면 그리는 데 낭비하지 않는 거죠.

04 이 과정의 선택

Day 1~3의 API가 그대로 백엔드입니다. 오늘 만드는 Streamlit은 그 API를 부르는 **여러 클라이언트 중 하나**일 뿐이에요 — 이 관점이 다음 장으로 이어집니다.

API 하나가 여러 얼굴을 동시에 상대한다

백엔드가 독립적으로 존재하면, 오늘 만드는 Streamlit은 **손님 중 한 명**이 됩니다. 모바일 앱·다른 서버·자동화 스크립트가 **같은 `/predict`**를 불러도 백엔드는 아무것도 바꿀 필요가 없어요 — 분리가 주는 가장 큰 보상입니다.



01 API는 공용 입구

REST API는 언어·플랫폼에 묶이지 않습니다. JSON으로 말하기만 하면 어떤 클라이언트든 같은 방식으로 호출해요 — Day 1에서 배운 '약속'의 힘입니다.

02 Streamlit은 그중 하나

오늘의 대시보드는 첫 번째 손님일 뿐입니다. 내일 모바일 앱이 붙고 모레 자동화 스크립트가 붙어도, 백엔드는 한 줄도 안 고칩니다.

03 변경도 따로따로

모델을 업데이트하면 FastAPI만 재배포, 화면을 바꾸면 Streamlit만 재배포합니다. 서로의 배포 일정에 발목 잡히지 않아요.

04 투자가 회수된다

잘 설계한 API 하나가 웹·앱·외부 파트너까지 두루 쓰입니다. Day 1~3에 들인 공이 클라이언트가 늘 때마다 다시 돌아오는 셈이죠.

오늘 완성되는 시스템의 전체 그림



Streamlit(8501) ↔ HTTP ↔ FastAPI(8000) ↔ 모델 파일 — 세 층의 분업.

01 Frontend — Streamlit · 8501

UI 그리기·파일 업로드·API 호출·결과 시각화를 맡습니다. 의존성은 `streamlit` 과 `requests` 정도 — **PyTorch는 설치조차 안 해도** 됩니다.

02 Backend — FastAPI · 8000

입력 검증·전처리·모델 추론·에러 처리·기록 — **핵심 로직 전부**가 여기 모입니다. Day 1~3에서 만든 그 서버 그대로예요.

03 Storage — 모델 파일

학습된 가중치는 FastAPI가 시작할 때 한 번 메모리에 올려 들고 있습니다. 프론트는 이 파일의 존재조차 모릅니다.

04 포트가 갈라놓는다

한 PC에서 두 프로그램이 같은 번호의 문(포트)을 쓸 수 없으니 8501과 8000으로 나눠 띄웁니다. 같은 PC든 다른 대륙의 서버든, **HTTP 주소만 맞으면** 연결돼요.

프론트는 클라이언트다 — requests로 부른다

Streamlit에서 FastAPI를 부르는 코드는 **Day 1에서 외부 API를 부르던 코드와 똑같습니다**. 달라진 건 주소뿐 — 남의 서버가 아니라 `localhost:8000`, 우리가 만든 서버예요. 입장이 바뀌어 우리가 '손님'이 된 겁니다.

```
import requests, base64

# GET — 서버 상태 확인 (Day 1에서 외부 API를 부르던 그 코드)
response = requests.get("http://localhost:8000/health")
health = response.json()          # {"status": "healthy", ...}

# POST — 추론 요청. 이미지는 Base64 '문자열'로 JSON에 담는다
image_base64 = base64.b64encode(image_bytes).decode("utf-8")
response = requests.post(
    "http://localhost:8000/predict/image",
    json={"image_base64": image_base64,
          "return_probabilities": True},
)
result = response.json()          # {"predicted_class": "7", ...}
```

01 GET /health 먼저

추론을 보내기 전에 서버가 살아 있는지부터 확인합니다. 대시보드 사이드바에 이 결과를 띄워, 사용자가 **호출 전에** 연결 상태를 가늠하게 해요.

02 추론은 POST + Body

Day 2에서 정한 약속 그대로입니다 — 크고 구조적인 입력은 본문 JSON에 담아 POST로 보냅니다. 이번엔 그 약속을 **서버가 아니라 클라이언트 입장**에서 지키는 거죠.

03 b64encode + decode

이미지 바이트를 Base64 문자열로 바꿔 JSON에 담습니다. 왜 이 수고가 필요한지는 §4.3에서 따로 풀어요.

04 .json()으로 마무리

응답 본문을 dict로 되돌려 예측값·확신도를 꺼냅니다. 이 dict를 화면에 그리는 게 §5 대시보드의 일입니다.

이미지 한 장이 결과가 되기까지 — 9단계 왕복



01 ①~③ 프론트가 보낸다

파일을 받아 Base64 문자열로 바꾸고, `POST /predict/image`의 JSON 본문에 담아 보냅니다. 여기까지가 오늘 우리가 새로 짜는 부분이에요.

02 ④~⑦ 백엔드가 일한다

검증 → Base64 복원 → 전처리 → 추론 → JSON 응답. 전부 Day 2~3에서 이미 만든 코드입니다 — 오늘은 손대지 않아요.

03 ⑧~⑨ 프론트가 그린다

응답 dict를 받아 숫자·확률 막대로 시각화합니다. 사용자는 9단계 왕복을 모른 채 결과만 봅니다.

04 경계가 또렷하다

이 흐름 어디에도 Streamlit이 텐서를 만지는 곳은 없습니다. 프론트는 **HTTP와 JSON**만 알면 돼요 — 그래서 백엔드 내부가 바뀌어도 프론트는 한 줄도 안 고칩니다.

Streamlit이 보내고, FastAPI가 추론하고, Streamlit이 그린다.

왜 이미지를 '글자'로 바꿔 보낼까

JSON은 텍스트 형식인데 이미지는 **이진(binary)** 데이터입니다 — 이 간극을 메우는 다리가 Base64예요.

JSON엔 바이트를 못 넣습니다. JSON 값은 문자열·숫자·참거짓·배열·객체뿐이에요. 이미지 파일의 원시 바이트를 그대로 끼우면 **유효한 JSON이 아니게** 됩니다.

Base64는 바이트를 글자로 옮깁니다. 3바이트를 안전한 영문·숫자 문자 4개로 바꾸는 규칙이에요. 어떤 이진 데이터든 **텍스트 어디에나** 끼워 넣을 수 있게 됩니다.

대가는 약 33%의 부피 증가입니다. 3바이트가 4글자가 되니까요. 작은 MNIST 이미지엔 티도 안 나지만, 수십 MB 파일이라면 파일 전송 전용 방식(폼 업로드)이 낫습니다.

`.decode("utf-8")` **까지 해야 끝입니다.** 인코딩 함수가 돌려주는 건 아직 **바이트열**이에요. `.decode()` 로 진짜 문자열로 바꿔야 JSON에 들어갑니다 — 이 한 단계를 빼먹으면 에러로 멈춥니다.

받는 쪽이 정확히 되돌립니다. FastAPI는 디코딩으로 바이트를 복원해 이미지·텐서로 변환합니다. 인코딩과 디코딩이 짝을 이뤄야 그림이 다시 그림이 돼요 — 파일 업로드 심화(Day 6)에서 이 패턴을 또 만납니다.

실패를 에러 화면 대신 안내 문장으로

API 호출은 **반드시 실패할 수 있습니다** — 서버가 꺼져 있거나, 느리거나, 에러 코드를 돌려주거나. 실패의 종류를 나눠 잡고, 사용자에게 **다음 행동**이 보이는 메시지를 띄웁니다.

```
import streamlit as st, requests

def call_api(url, json_data=None, method="post"):
    """성공하면 dict, 실패하면 None을 돌려준다."""
    try:
        if method == "get":
            resp = requests.get(url, timeout=10)
        else:
            resp = requests.post(url, json=json_data, timeout=30)
            resp.raise_for_status() # 4xx/5xx면 예외 발생
        return resp.json()
    except requests.exceptions.ConnectionError:
        st.error("서버에 연결할 수 없습니다. FastAPI 실행을 확인하세요.")
    except requests.exceptions.Timeout:
        st.warning("응답 시간 초과 — 잠시 후 다시 시도하세요.")
    except requests.exceptions.HTTPError as e:
        st.error(f"서버 에러 (HTTP {e.response.status_code})")
    except Exception as e:
        st.error(f"오류: {type(e).__name__}")
    return None
```

01 ConnectionError — 꺼져 있다

백엔드가 안 떠 있을 때입니다. "서버에 연결할 수 없습니다. FastAPI 실행을 확인하세요"처럼 **무엇을 하면 되는지**까지 알려줘야 좋은 안내예요.

02 Timeout — 너무 느리다

`timeout=`으로 기다림의 한계를 정합니다. 안 정하면 응답 없는 서버를 **무한정** 기다리며 앱이 굳어요 — 한계를 넘으면 재시도를 권합니다.

03 raise_for_status — 답은 왔는데 에러

연결은 됐지만 422·500처럼 실패 코드가 온 경우 예외를 일으킵니다. Day 2에서 배운 상태 코드가 여기서 **클라이언트의 분기 기준**이 되죠.

04 성공 dict, 실패 None

이 함수의 계약입니다. 호출하는 쪽은 `if result:` 한 줄로 성공·실패를 갈라요 — 실패 안내는 함수 안에서 이미 끝났으니까요.

사용자에게 에러의 '속살'을 보여주지 않는다

같은 실패라도 **어떻게 알리느냐**가 사용자 경험과 보안을 가릅니다.

UX

사용자는 행동을 원한다

`KeyError: 'image'` 같은 내부 오류 문구는 사용자에게 소음일 뿐입니다. "서버에 연결할 수 없습니다 — 잠시 후 다시 시도하세요"처럼 **다음 행동이 보이는 문장**으로 바꿔 말해야 해요.

보안

속살은 단서가 된다

내부 오류 화면엔 파일 경로·라이브러리 버전·코드 구조가 담깁니다. 공격자에게 지도를 쥐여주는 셈이라, 화면엔 절대 띄우지 않습니다.

로그

원인은 서버에 남긴다

상세한 원인은 Day 3에서 만든 **서버 기록**에 이미 남아 있습니다. 디버깅은 기록으로, 안내는 메시지로 — 역할을 나누면 둘 다 잘하게 돼요.

오늘 배운 조각이 한 파일에 모인다

이제 합칩니다 — 업로드·캐시·API 호출·에러 처리가 대시보드 하나로 완결돼요. `layout="wide"` 로 화면을 넓게 펴고, 사이드바엔 상태, 본문은 컬럼 둘로 **입력 | 결과**를 나란히 둡니다.

```
import streamlit as st, base64
st.set_page_config(page_title="MNIST 대시보드", layout="wide")

with st.sidebar:
    # 설정·서버 상태는 사이드바로
    health = call_api(f"{API}/health", method="get")
    st.write("서버 연결됨" if health else "서버 꺼짐 - 백엔드 확인")
col1, col2 = st.columns(2)
    # 왼쪽 입력 | 오른쪽 결과
with col1:
    up = st.file_uploader("이미지", type=["png", "jpg"])
    run = st.button("추론 실행")
if run and up:
    # 버튼 + 파일, 둘 다 있을 때만
    b64 = base64.b64encode(up.getvalue()).decode()
    with st.spinner("추론 중..."):
        res = call_api(f"{API}/predict/image", {"image_base64": b64})
    if res:
        # 성공 시에만 - 재실행에도 살아남게
        st.session_state["last"] = res
with col2:
    if r := st.session_state.get("last"):
        st.metric("예측 숫자", r["predicted_class"])
```

01 sidebar — 상태와 설정의 자리

`with st.sidebar:` 블록 안의 요소는 왼쪽 사이드바에 그려집니다.

`/health` 결과를 여기 띄워, 사용자가 추론 버튼을 누르기 **전에** 서버 상태를 알게 해요.

02 columns — 입력과 결과를 나란히

`st.columns(2)` 가 화면을 좌우로 나눕니다. 왼쪽에서 올리고 오른쪽에서 확인하는, 대시보드의 가장 흔한 구성이에요.

03 run and up — 둘 다 있을 때만

버튼이 눌렸고(`run`) 파일도 있을 때(`up`)만 호출합니다. Base64
→ `spinner` (기다림 표시) → `call_api` — §2~§4의 패턴이 그대로 이어지죠.

04 session_state에 결과 보관

응답을 `st.session_state["last"]` 에 담습니다. 안 담으면 어떻게 되냐고요? — 바로 다음 장에서 그 사고를 봅니다.

추론 결과가 자꾸 사라진다 — 누가 지웠을까

범인은 §1.3의 재실행입니다. 버튼을 눌러 결과를 받아도, 체크박스 하나만 건드리면 스크립트가 다시 돌며 **변수가 전부 새로** 만들어져요. `st.session_state` 는 재실행이 지우지 못하는 **딕셔너리형 저장소** — 여기 담은 값만 살아남습니다. 쓰는 패턴은 셋: **없을 때만 초기화** → **이벤트에서 갱신** → **매 재실행마다 표시**.



01 나쁜 예 — 변수에 결과

`result = call_api(...)` 로 받고 끝내면, 다음 재실행에서 `result` 는 존재하지 않습니다. 사용자 눈엔 **방금 본 결과가 증발**하는 버그로 보여요.

02 좋은 예 — session_state에 저장

`st.session_state["last"] = result` 로 담아두면, 재실행 후에도 꺼내서 다시 그릴 수 있습니다. 버튼이 False로 돌아가도 결과는 화면에 남아요.

03 초기화는 '없을 때만'

`if "count" not in st.session_state:` 검사가 핵심입니다. 이 검사 없이 매번 0을 대입하면 **재실행마다 리셋** — 기억하려던 값을 스스로 지우는 꼴이죠.

04 캐시와는 역할이 다르다

캐시가 **비용**(객체 재생성)을 줄인다면, session_state는 **망각**(결과 증발)을 막습니다. 참고로 LLM 챗봇(Day 7)의 대화 기록도 정확히 이 패턴이에요.

켜고, 보내고, 끊어 본다

서버 둘을 **각자 터미널**에 띄웁니다 — 떠 있는 동안 터미널을 점유하니 창도 두 개. 그리고 잘 되는 것만이 아니라 **잘 끊기는지**까지 확인해야 합격입니다.

STEP 01

백엔드 실행

```
uvicorn app.main_final:app
--port 8000
```

— Day 3까지
완성한 추론 서버를 먼저 띄웁니다.



STEP 02

프론트 실행

다른 터미널에서

```
streamlit run
frontend/app_dashboard.py
--server.port 8501
```

 —
브라우저로 접속합니다.



STEP 03

기본 추론

사이드바 상태가 '연결됨'인지 보고,
샘플 이미지로 추론합니다 — 예측
숫자와 확률 막대가 떠야 정상이에요.



STEP 04

파일 업로드

임의의 숫자 이미지를 드래그 앤
드롭해 같은 흐름을 확인합니다. 짧게
학습한 모델이라 종종 틀려요 —
그래도 UI는 완벽히 일합니다.



STEP 05

에러 상황

백엔드를 일부러 끕니다. 상태가
'꺼짐'으로 바뀌고 추론은 친절하
에러로 막히면 — §4의 방어가
동작한 겁니다.

프로토타입엔 최고 — 그런데 언제 졸업할까?

도구의 한계를 알아야 **언제 다음 도구로 넘어갈지** 판단할 수 있습니다. 다행히 백엔드는 어느 쪽이든 그대로예요.

잘 맞는 곳

Streamlit — 데모·내부 도구

압도적인 속도

Python만으로 수십 분이면 동작하는 화면이 나옵니다. 데모·사내 대시보드·실험 공유에선 대체재가 없어요.

재실행 모델의 청구서

인터랙션마다 스크립트가 통째로 도는 구조라, 동시 사용자가 많아지면 서버 자원이 빠르게 바닥납니다. 수백 명이 쓰는 서비스라면 한계가 먼저 와요.

디자인의 천장

레이아웃·스타일 자유도가 낮습니다. 정해진 틀 안에서만 움직여서, 브랜드 디자인이나 픽셀 단위 UX가 필요한 제품 화면엔 닿지 않아요.

넘어갈 때

React + FastAPI — 제품 단계

프론트만 교체한다

오늘 만든 FastAPI는 그대로 두고 화면만 React로 바꿉니다. §3에서 분리해 둔 덕에 **백엔드 자산은 전부 재사용**돼요.

고트래픽·세밀한 UX

상태를 브라우저가 관리해 서버 부담이 적고, 디자인·인터랙션을 자유롭게 만들 수 있습니다. 수천 명이 동시에 써도 견디는 화면은 이쪽이에요.

비용은 복잡도

프론트엔드 전문 지식과 빌드 환경이 필요합니다. 데모 단계에 미리 치를 필요는 없고, **제품 단계**에 이르렀을 때 치르는 값이에요.

개발자만 쓰던 API가, 누구나 쓰는 화면이 되다

오늘 한 일

- ▶ **Streamlit** — 파이썬 스크립트가 곧 웹앱, 클릭마다 **전체 재실행**
- ▶ **두 캐시** — 데이터는 복사(cache_data), 객체는 공유(cache_resource)
- ▶ **file_uploader** — None 검사 → 미리보기 → 원본 바이트
- ▶ **분리 아키텍처** — 프론트(8501)는 백엔드(8000)를 부르는 클라이언트
- ▶ **requests + Base64** 호출, 친절한 에러, **session_state**로 결과 유지

스크립트가 곧 웹앱이고, 매번 다시 실행된다 — 그래서 비용은 캐시로, 기억은 session_state로 지킨다. 화면은 모델을 모른 채 8000번의 API를 부를 뿐, 그렇게 모델 배포가 사용자의 손에 닿는다.

다음 — DAY 5

- ▶ **정형 데이터(CSV)** 예측 서비스를 처음부터 끝까지 구축
- ▶ Day 1~4의 모든 기술을 **하나의 프로젝트**로 통합
- ▶ 학습 → 저장 → API → 프론트엔드 **전 파이프라인** 연결
- ▶ 이미지 대신 **구조화된 입력** — 스키마 설계가 더 중요해진다
- ▶ 실제 서비스에 가까운 **완결된 프로젝트** 경험